Synthetic Cognitive Coding (SCC)

Engineering Handbook

Hardened Builder Edition

Implementation-First Field Guide for SCC-Governed Systems

Engineering hardening pass for implementers, reviewers, SREs, and reference-kernel builders

Practical companion to the formal constitutional-operational canon

Written by Ian Farquharson
Emergent-Evo Engineer | Neural-Synth Architect

Logo asset preserved by reference:
https://gist.github.com/user-attachments/assets/0b9fbc2b-ae9c-4a68-989c-84bae78849dc

Dedicated to my early mentor, Dr. Robert Whetsel, whose guidance first set me on this path.
LinkedIn: https://www.linkedin.com/in/robertwhetsel

Special thanks to my friend Vlad Puscoiu, whose friendship over the years has helped keep my work aligned with my goals and aspirations. He was the person who first taught me how to open a terminal.
LinkedIn: https://nl.linkedin.com/in/vlad-puscoiu-152436246

Handbook version date: April 29, 2026
Companion source canon date: April 26, 2026

This handbook is an implementation guide. The formal SCC canon remains the controlling source for theorem statements, constitutional clauses, proof obligations, and claim boundaries.

Hardened builder edition note: this adaptation preserves the original handbook and adds implementation guardrails, repository setup, schema and transaction rules, CI/CD gates, security hardening, and runbook material.

Abstract

This hardened builder edition turns the SCC formal canon into an implementation plan and an engineering setup, with an explicit handoff path to Kernel Fortress-style executable compliance gates. It is written for engineers who need to build, audit, test, optimize, operate, and ship SCC-governed systems without losing the formal discipline of the canon. The focus is practical: what to build first, what must never be bypassed, how to structure state, how to encode events, how to generate proof-obligation bundles, how to check them, how to replay a run, how to treat stuttering, how to instrument drift, how to secure the runtime boundary, how to define transactional storage, how to wire CI/CD release gates, and how to avoid the common mistakes that make a compliance story collapse. The handbook is shorter, more direct, and more operational than the scholarly specification. It does not replace the canon. It translates the canon into buildable, testable, reviewable engineering work.

Reader Contract

This document assumes you are implementing SCC, reviewing an SCC implementation, or converting the formal canon into a working kernel. It deliberately uses short sentences, explicit gates, and practical examples. When there is any conflict between this handbook and the formal canon, the canon controls.

Contents

How to Use This Handbook

Read this in build order. Do not start with federation. Do not start with optimization. Do not start with a UI. Build the kernel first.

Shipping gate. A system is not SCC-governed because it has a policy prompt, a checklist, or a safety wrapper. It is SCC-governed only when ordinary execution steps are typed, checked, witnessed, audit-legible, lineage-preserving, and rejected or halted when the proof-obligation bundle fails.

Who should read which sections

Reader	Read first
Kernel engineer	Build Order; Minimum Data Model; PO Bundle; Seven-Stage Step Function.
Verifier engineer	Checker First, Optimizer Second; Test Harness; Mechanization Priorities.
Runtime engineer	Canonical Event Encoding; Audit and Replay; Performance Guide.
Safety or governance reviewer	Engineering Contract; Governance and Risk; Deployment Readiness.
Distributed-systems engineer	Federation and Quorum Safety, but only after the single-node kernel is stable.

One sentence

SCC makes legality a runtime invariant, not a post-hoc explanation.

The handbook rule

If a step cannot produce a valid proof-obligation bundle, the step is not merely suspicious. It is ill-formed.

Engineering Contract

The formal canon has three orders: constitutional, operational-mathematical, and meta-formal. The implementation must respect all three.

- The constitutional order says what must never be violated.
- The operational order says how state evolves.
- The meta-formal order says how implementation steps are witnessed, classified, and lifted back to the abstract specification.

In code, this means four things.

1. Every state has a declared type.
2. Every ordinary step has a declared transition path.
3. Every step emits a proof-obligation bundle.
4. Every bundle is checked before the step is accepted.

Hard rule. Fail closed. If the checker is unavailable, corrupted, stale, or ambiguous, do not continue ordinary execution.

The four non-negotiables

Non-negotiable	Engineering meaning	Common failure
Typed state	Every coordinate has a declared domain and serializer.	Storing mixed ad-hoc dictionaries and hoping tests catch drift.
Hard clauses	Protected rules are checked before ordinary compute is accepted.	Treating hard clauses as advisory policy text.
Proof-obligation bundle	Each step has machine-checkable evidence.	Writing logs but not executable witnesses.
Replay	Same state, event, build, and deterministic mode reproduce the same abstract trajectory.	Hashing non-canonical data or depending on wall-clock randomness.

The System You Are Building

An SCC implementation is a small constitutional runtime around a cognitive system. The cognitive system can be large. The SCC kernel should not be.

Minimal architecture

Listing 1: Minimal SCC runtime shape.

```
class SCCRuntime:
    def __init__(self, constitution, checker, audit_log, store):
        self.constitution = constitution
        self.checker = checker
        self.audit_log = audit_log
        self.store = store

    def step(self, state, raw_input, env):
        event = canonicalize_input(raw_input, env)
        candidate, witnesses = seven_stage_transition(state, event, env)
        po = build_bundle(state, event, candidate, witnesses, env)

        if not self.checker.accepts(po):
            return enter_halt(state, event, reason="po_rejected")

        self.audit_log.append(po.audit_event)
        self.store.commit(candidate, po)
        return candidate
```

What belongs in the SCC kernel

- State schema.
- Hard-clause registry.

- Canonical event encoder.
- Seven-stage step executor.
- Proof-obligation bundle builder.
- Witness checker.
- Audit hash update.
- Replay harness.
- Halt and recovery gate.

What should stay outside the kernel

- Model weights.
- User interface logic.
- Product telemetry dashboards.
- Convenience routing.
- Non-critical caching.
- Long-running analytics jobs.

The kernel should be boring. Boring is good. Boring can be audited.

Runtime Boundaries and TCB Hardening

The kernel is small, but it is still a trusted system. Name the trusted pieces. Keep their interfaces narrow. Make every shortcut visible in the TCB ledger.

Dependency direction

A hardened implementation has one-way dependencies.

Listing 2: Allowed dependency direction.

```
events -> transition -> witnesses -> po_bundle -> checker -> store_commit -> output
schemas -> serializers -> hashes -> checker
constitution -> hard_clause_registry -> checker
```

The checker may depend on schemas, canonical serializers, hash functions, hard-clause definitions, and witness readers. It must not depend on the transition implementation that produced the candidate.

Hard rule. Do not let the checker import product routing, model execution, prompt construction, telemetry clients, live network clients, or mutable runtime containers. The checker is a verifier, not a participant in the run.

Trusted computing base ledger

TCB item	Trusted for	Hardening expectation
Canonical serializer	Stable event, state, bundle, and audit bytes.	Golden vectors across machines; no object reprs; schema id included.

Hash and signature library	Audit chain, state identity, certificates.	Pinned versions; domain separation; signature test vectors.
Checker	Bundle rejection and acceptance.	Side-effect free; deterministic; negative corpus; soundness target.
Storage transaction layer	Atomic state, bundle, and audit commit.	Single commit boundary; idempotent step ids; crash-recovery tests.
Clock and randomness adapter	Deterministic replay boundary.	No direct wall-clock or random calls in kernel; recorded inputs only.
Compiler and runtime	Executes trusted kernel code.	Version pinned in release manifest; reproducible build notes.

Step result contract

Do not return raw model output from the transition path. Return an explicit step result after the checker and store commit have completed.

Listing 3: Hardened step result contract.

```
from dataclasses import dataclass
from enum import Enum

class StepStatus(str, Enum):
    ACCEPTED = "accepted"
    REJECTED = "rejected"
    HALTED = "halted"

@dataclass(frozen=True)
class StepResult:
    status: StepStatus
    state_hash: str
    audit_hash: str
    po_hash: str | None
    output_ref: str | None
    reason: str | None

# Output is only reachable when status == ACCEPTED and the commit succeeded.
```

Dangerous imports and capabilities

Capability	Kernel rule
Network calls	Forbidden in checker and canonicalization. Use recorded events or signed inputs.
Wall-clock time	Forbidden in replay-sensitive logic. Time must enter as a canonical event field.
Randomness	Forbidden in checker. Allowed only when seed, source, and numerical mode are recorded.
Mutable globals	Forbidden in checker. Treat global caches as outside the trusted path.
Environment variables	Not a source of law. Descriptor values must be committed and hashed.
Product telemetry	Never a proof source. Telemetry can observe; it cannot authorize.

Shipping gate. A builder-ready kernel has a TCB ledger in the repository. Every trusted dependency has an owner, version, reason for trust, test vector, and replacement plan.

Builder Quickstart

This section is for the engineer opening a blank repository. The goal is not to build the whole cognitive system. The goal is to build the smallest SCC kernel that can reject a bad step and replay a good step.

Repository shape

Start with a boring reference layout. Keep product code outside the kernel.

Listing 4: Day-one repository layout.

```
scc-reference/
  pyproject.toml
  Makefile
  src/scc_kernel/
    state.py
    events.py
    audit.py
    po.py
    checker.py
    transition.py
    runtime.py
    store.py
    replay.py
  tests/
    test_negative_corpus.py
    test_replay.py
    test_halt.py
    test_protected_state.py
  fixtures/
    normal_trace.json
    rejected_trace.json
  docs/
    deployment_descriptor.yaml
    hard_clauses.yaml
    tcb_ledger.md
    runbook.md
```

First runnable slice

Build a vertical slice before adding intelligence.

1. Define `SCCState` with canonical serialization and golden hash vectors.
2. Define one canonical event type, one benign event fixture, and one malicious event fixture.
3. Define `POBundle` and make missing fields unrepresentable where possible.
4. Build a checker that accepts one valid fixture and rejects the negative corpus.
5. Build a transition that can only stutter, halt, or perform a tiny deterministic memory update.
6. Commit state, PO bundle, and audit event atomically.
7. Replay the valid trace twice and compare state hashes.
8. Run the malicious trace and prove output cannot escape.

Make targets

Every builder should be able to run the same gates locally and in CI.

Listing 5: Minimum make targets.

```
make test        # unit tests and negative corpus
make replay      # deterministic replay fixtures
make verify      # schemas, checker, audit chain, replay, lint
make package     # release manifest and artifact bundle
```

Engineer handoff checklist

Artifact	Builder-ready condition
README	Explains the runtime contract, commands, and first trace to run.
Descriptor	Pins build id, checker version, numerical mode, schema ids, and active hard clauses.
Golden vectors	Include canonical bytes and hashes for state, event, PO, and audit event.
Negative corpus	Fails for every known bad pattern in this handbook.
Replay fixtures	Include accepted, rejected, halted, stutter, and recovery-meta-path traces.
Runbook	Tells operators what to do when checker, audit, replay, or lineage fails.
TCB ledger	Names trusted dependencies, versions, owners, and test evidence.

Shipping gate. The first engineering milestone is not a smart agent. It is a kernel that can prove it refuses to be a dumb illegal one.

Build Order

Build SCC in this order. Changing the order usually creates fake confidence.

1. Define the state schema.
2. Define the hard-clause schema.
3. Define canonical event encoding.
4. Define audit hash update.
5. Define the proof-obligation bundle.
6. Build the checker.
7. Build the seven-stage transition.
8. Add replay.
9. Add drift metrics.
10. Add governance and risk updates.

11. Add memory and lineage operations.
12. Add ρ and ρ_b extraction.
13. Add negative tests.
14. Add performance instrumentation.
15. Add federation only after the single-node system survives adversarial tests.

Failure mode. Building the agent first and bolting SCC on later usually produces a wrapper, not a constitutional runtime.

Implementation milestones

Milestone	Done when	Do not move on if
M0: State	Every coordinate has a type, serializer, and test fixture.	Protected state can be mutated by ordinary code.
M1: Events	Canonical event bytes are stable across machines.	Hashes differ between equivalent runs.
M2: PO bundle	The bundle has all required fields and witness references.	Fields are comments or strings instead of executable booleans.
M3: Checker	Accepted bundles imply the required predicates in tests and proof targets.	The checker trusts the component that produced the bundle.
M4: Step	Every ordinary step goes through seven stages.	Any shortcut can emit output without a checked bundle.
M5: Replay	A recorded run can be replayed deterministically.	Replay depends on live services, wall-clock time, or nondeterministic serialization.
M6: Stress	Negative tests force rejection or halt.	The happy path is the only tested path.

Minimum Data Model

Start with one explicit state record. Keep it boring. Make every field serializable.

Abstract state fields

Listing 6: Minimal state record.

```
from dataclasses import dataclass
from typing import Mapping, Tuple

Hash = str
LineageId = str
ProtectedRoot = str

@dataclass(frozen=True)
class SCCState:
    compute_ref: str              # h: reference to compute state
    memory_root: Hash             # M: content-addressed memory root
```

```
    vector_clock: Mapping[str, int]
    governance_weights: Tuple[float, ...]
    failure_mode: int             # 0, 1, 2, or 3. 3 means halt.
    audit_hash: Hash              # H_t
    risk_vector: Tuple[float, ...]
    lineage_id: LineageId
    protected_root: ProtectedRoot # p: ordinary code must not mutate this
```

Protected coordinate rule

The protected coordinate is not just another field. It is the sovereignty membrane.

Hard rule. No ordinary transition may mutate `protected_root`. Only an explicitly authorized exception or amendment path may touch it.

Practical invariant checks

Listing 7: Basic invariant checks.

```
def valid_failure_mode(mode: int) -> bool:
    return mode in {0, 1, 2, 3}

def valid_simplex(weights: tuple[float, ...], eps: float = 1e-9) -> bool:
    return all(w >= -eps for w in weights) and abs(sum(weights) - 1.0) <= eps

def protected_unchanged(prev: SCCState, nxt: SCCState, authorized: bool) -> bool:
    if authorized:
        return True
    return prev.protected_root == nxt.protected_root

def halt_absorbed(prev: SCCState, nxt: SCCState, recovery_authorized: bool) -> bool:
    if prev.failure_mode == 3 and not recovery_authorized:
        return nxt.failure_mode == 3
    return True
```

Canonical Event Encoding

Replay dies when encoding is sloppy. Fix it early.

Rules

1. Encode to UTF-8 bytes.
2. Sort object keys.
3. Normalize timestamps.
4. Avoid raw floats in canonical records unless the numerical mode is fixed.
5. Include build id, descriptor id, and schema version.
6. Domain-separate every hash.

Listing 8: Canonical JSON encoder.

```
import json
```

```
import hashlib

DOMAIN = b"SCC-AUDIT-V1\x00"

def canonical_bytes(obj: dict) -> bytes:
    return json.dumps(
        obj,
        sort_keys=True,
        separators=(",", ":"),
        ensure_ascii=False,
    ).encode("utf-8")

def audit_hash(prev_hash: str, event: dict) -> str:
    h = hashlib.sha256()
    h.update(DOMAIN)
    h.update(bytes.fromhex(prev_hash))
    h.update(canonical_bytes(event))
    return h.hexdigest()
```

Do this, not that

Do this	Not that	Why
Hash canonical bytes.	Hash Python object reprs.	Object reprs are not stable enough for replay.
Use domain tags.	Reuse a bare hash function everywhere.	Domain tags prevent accidental cross-protocol ambiguity.
Record schema version.	Infer schema from context.	Context disappears during audit.
Freeze numerical mode.	Let each machine choose float behavior.	Drift measurements become non-reproducible.

Schema, Versioning, and Migration Discipline

Schema drift is one of the fastest ways to lose replay. Treat schemas as runtime law, not documentation.

Schema contract

Every canonical object must carry enough information for another implementation to parse, hash, and reject it without oral history.

- State records carry a state schema id.
- Events carry an event schema id, build id, descriptor id, and producer id.
- PO bundles carry a PO schema id and checker version.
- Audit events carry an audit schema id and hash domain.
- Descriptors carry all active schema ids and numerical mode.

Hard rule. A field added without a schema bump is a replay break. A field removed without a migration event is an audit break.

Schema registry

Listing 9: Minimal schema registry entry.

```
schema: scc.schema.registry.v1
schemas:
  scc.state.v1:
    canonicalization: canonical-json-v1
    hash_domain: SCC-STATE-V1
    owner: kernel
  scc.event.v1:
    canonicalization: canonical-json-v1
    hash_domain: SCC-EVENT-V1
    owner: runtime
  scc.po.v1:
    canonicalization: canonical-json-v1
    hash_domain: SCC-PO-V1
    owner: checker
```

Wire object envelope

Put schema, descriptor, and build identity at the envelope. Do not bury them in free-form payloads.

Listing 10: Canonical event envelope.

```
{
  "schema": "scc.event.v1",
  "step_id": "node-a:000001",
  "build_id": "build:abc123",
  "descriptor_id": "theta:prod-a",
  "producer": "gateway-a",
  "payload_type": "user_input",
  "payload_hash": "...",
  "payload": {"kind": "text", "body": "..."}
}
```

Migration rules

1. Migrations are typed events, not silent deserializers.
2. Migration input and output hashes are recorded.
3. The checker accepts a migrated bundle only if the descriptor names the migration path.
4. A migration that touches protected state must use an authorized meta-path.
5. Old replay fixtures remain in the test suite until their migration proof is checked.

Compatibility table

Change	Compatibility	Required action
Add optional non-semantic field	Sometimes compatible.	Schema bump; canonical default; golden vector.
Rename field	Breaking.	Migration event and replay fixture.
Change hash domain	Breaking.	New descriptor and genesis or declared migration.

Change numerical mode	Breaking for metrics.	New descriptor, drift baselines, replay fixtures.
Change checker logic	Potentially breaking.	Checker version bump and negative corpus review.

Proof-Obligation Bundle

The proof-obligation bundle is the central engineering artifact. Treat it like the syscall boundary of SCC.

Minimum bundle

Listing 11: Proof-obligation bundle.

```
from dataclasses import dataclass
from enum import Enum
from typing import Any

class StepMode(str, Enum):
    EXACT = "Exact"
    STUTTER = "Stutter"
    SAFE_REFINED = "SafeRefined"

@dataclass(frozen=True)
class POBundle:
    schema_version: str
    step_id: str
    mode: StepMode

    dom: bool
    inv: bool
    identity: bool
    governance: bool
    risk: bool
    audit: bool
    isolation: bool
    refinement: bool

    prev_state_hash: str
    next_state_hash: str
    event_hash: str
    audit_event: dict
    witness_refs: dict[str, Any]
    checker_version: str
```

Bundle rule

Hard rule. The output is not accepted until the bundle is accepted. Logs are not enough. Comments are not enough. A bundle is evidence only if the checker can evaluate it.

Bundle fields

Field	Meaning	Reject when
`dom`	State is in the admissible domain.	Missing required coordinate, invalid mode, invalid simplex.
`inv`	Hard invariants still hold.	Any active hard clause fails.
`identity`	Identity continuity is preserved.	Lineage rupture, protected mutation, or illegal drift.
`governance`	Governance update was lawful.	Update bypasses projection or legitimacy gate.
`risk`	Risk law and halt law were applied.	Halt threshold is crossed without halt.
`audit`	Audit event and hash update are canonical.	Non-canonical bytes or broken hash chain.
`isolation`	Protected coordinate is untouched unless authorized.	Ordinary path changes protected state.
`refinement`	Step classification is valid.	Exact, Stutter, and SafeRefined all fail or overlap improperly.

Checker First, Optimizer Second

The checker is the compliance gate. It should be small. It should be boring. It should not trust the producer.

Listing 12: Fail-closed checker skeleton.

```
REQUIRED_TRUE_FIELDS = [
    "dom", "inv", "identity", "governance", "risk",
    "audit", "isolation", "refinement",
]

def accepts_bundle(po: POBundle, prev: SCCState, nxt: SCCState, env) -> bool:
    if po.schema_version != env.expected_po_schema:
        return False
    if po.checker_version != env.checker_version:
        return False
    if any(getattr(po, field) is not True for field in REQUIRED_TRUE_FIELDS):
        return False
    if not verify_state_hash(po.prev_state_hash, prev):
        return False
    if not verify_state_hash(po.next_state_hash, nxt):
        return False
    if not verify_audit_event(po.audit_event, prev, nxt, env):
        return False
    if not verify_step_mode(po.mode, prev, nxt, env):
        return False
    return True
```

Checker design

- Keep the checker deterministic.
- Keep it side-effect free.
- Keep it versioned.

- Keep it separately testable.
- Treat every accepted shortcut as part of the trusted computing base.

Failure mode. The most dangerous checker is one that reads the same mutable runtime objects produced by the transition path. The checker should verify committed candidate artifacts, canonical bytes, and witness references.

Boolean-to-predicate boundary

The formal canon distinguishes executable booleans from mathematical propositions. Engineering translation: a boolean flag is not a proof by itself. It is a result from a checker with a declared soundness argument.

Listing 13: Lean-style target for checker soundness.

```
theorem checker_sound
  (po : POBundle)
  (h : checker_accepts po = true) :
  Dom po /\ Inv po /\ Identity po /\ Audit po /\ Refinement po := by
  -- The implementation proof must connect executable checks to predicates.
  -- Until this theorem exists, checker soundness remains a TCB item.
  admit
```

The Seven-Stage Step Function

Every ordinary step factors through seven stages. Do not let product code bypass them.

1. Input and observation intake.
2. Constitutional and hard-clause check.
3. Governance and risk update.
4. Core compute and shard coupling.
5. Memory and lineage operation.
6. Audit and administrative advancement.
7. Refinement classification and output emission.

Implementation skeleton

Listing 14: Seven-stage transition skeleton.

```
def seven_stage_transition(state: SCCState, event: dict, env):
    witnesses = {}

    s1, witnesses["intake"] = stage_1_intake(state, event, env)
    s2, witnesses["clauses"] = stage_2_hard_clause_check(s1, env)
    s3, witnesses["gov_risk"] = stage_3_governance_risk(s2, event, env)
    s4, witnesses["compute"] = stage_4_core_compute(s3, event, env)
    s5, witnesses["memory"] = stage_5_memory_lineage(s4, event, env)
    s6, witnesses["audit"] = stage_6_audit_admin(s5, event, env)
    s7, witnesses["refinement"] = stage_7_refinement_output(s6, event, env)

    return s7, witnesses
```

Stage outputs

Each stage should return a candidate state and a typed witness. The witness must be small enough to check. Do not return arbitrary prose.

Listing 15: Typed witness example.

```
@dataclass(frozen=True)
class GovernanceWitness:
    legitimacy_ok: bool
    projected_to_simplex: bool
    drift_bound: float
    previous_weights_hash: str
    next_weights_hash: str
```

Hard rule. Stage witnesses are not audit logs. They are inputs to the proof-obligation bundle. Logs explain. Witnesses verify.

State Metrics and Drift Tracking

SCC uses metrics to reason about non-protected state. Metrics do not authorize protected-coordinate changes.

Engineering metric

Start with a weighted product metric over non-protected coordinates.

Listing 16: Weighted drift calculation.

```
def indicator(a, b) -> int:
    return 0 if a == b else 1

def drift(prev: SCCState, nxt: SCCState, weights) -> float:
    return (
        weights["compute"] * compute_distance(prev.compute_ref, nxt.compute_ref)
        + weights["memory"] * memory_distance(prev.memory_root, nxt.memory_root)
        + weights["clock"] * l1_clock(prev.vector_clock, nxt.vector_clock)
        + weights["governance"] * l1(prev.governance_weights, nxt.governance_weights)
        + weights["failure"] * indicator(prev.failure_mode, nxt.failure_mode)
        + weights["risk"] * l1(prev.risk_vector, nxt.risk_vector)
        + weights["lineage"] * indicator(prev.lineage_id, nxt.lineage_id)
    )
```

Metric gotchas

- Do not include the protected coordinate in ordinary contraction claims unless the formal model explicitly allows it.
- Do not use a metric value as a legality certificate.
- Do not compare drift across builds unless weights and coordinate metrics are pinned.
- Do not hide metric weights in code constants. Put them in the execution descriptor.

When a drift spike matters

A drift spike matters when it changes risk, violates a bound, indicates lineage rupture, or invalidates a contraction/stability assumption. A spike does not automatically prove illegality. It is evidence. The checker still decides lawfulness.

Governance, Risk, and Halt

Governance is state. Risk is state. Halt is state. Treat them that way.

Governance update

Governance weights live in a simplex. A lawful ordinary update projects back into the simplex and records the legitimacy witness.

Listing 17: Simplex projection sketch.

```
def project_simplex(v):
    # Production code should use a reviewed numerical routine.
    # This sketch is for shape only.
    n = len(v)
    u = sorted(v, reverse=True)
    cssv = [sum(u[: i + 1]) - 1.0 for i in range(n)]
    rho = max(i for i in range(n) if u[i] - cssv[i] / (i + 1) > 0)
    theta = cssv[rho] / (rho + 1)
    return tuple(max(x - theta, 0.0) for x in v)

def governance_step(w, proposal, eta, legitimacy_ok):
    if not legitimacy_ok:
        return w
    return project_simplex([wi + eta * gi for wi, gi in zip(w, proposal)])
```

Risk update

Risk must be explicit. Do not hide risk in a log message.

Listing 18: Risk-to-halt gate.

```
def apply_risk_law(state: SCCState, risk_event, env) -> SCCState:
    r_next = env.risk_model.update(state.risk_vector, risk_event)
    mode_next = state.failure_mode

    if env.risk_model.requires_halt(r_next):
        mode_next = 3
    elif env.risk_model.requires_escalation(r_next):
        mode_next = max(mode_next, 2)

    return replace(state, risk_vector=r_next, failure_mode=mode_next)
```

Halt rule

Hard rule. Failure mode 3 is absorbing for ordinary execution. Recovery is not an ordinary step. It is a constitutionally authorized meta-path.

Halt decision tree

1. Is the current state already in halt? If yes, stay halted unless recovery is authorized.
2. Did risk cross a halt threshold? If yes, enter halt.
3. Did the PO checker reject? If yes, enter halt or reject the candidate according to the constitution.
4. Did the protected coordinate change without authorization? If yes, enter halt.
5. Otherwise, continue ordinary execution.

Security and Abuse Hardening

SCC hardening is not only about mathematical checks. It is also about adversarial inputs, compromised services, operational mistakes, and abuse of gray areas.

Threat model

Threat	Likely attack	Required control
Forged event	Submit input with false descriptor or step id.	Signed ingress, canonical envelope, replayed hash check.
Checker bypass	Emit product output before proof acceptance.	StepResult contract, output gate, integration test.
Protected-state mutation	Ordinary code rewrites protected root.	Immutable state record, isolation check, meta-path only.
Audit tampering	Edit or delete audit records.	Append-only storage, hash chain verification, signed checkpoints.
Replay poison	Depend on live services or wall-clock time.	Dependency injection, recorded events, deterministic mode.
Resource exhaustion	Huge input, oversized witness, recursive lineage.	Input limits, witness size limits, timeout budget, halt escalation.
Governance capture	Illegitimate proposal changes weights.	Legitimacy witness, simplex projection, drift bound.

Input hardening

1. Reject inputs that do not parse into a known event schema.
2. Apply size limits before canonicalization and before model execution.
3. Normalize once and record the canonical bytes.
4. Record payload hashes for large payloads. Store payloads in content-addressed storage.
5. Treat parser errors as rejection, not partial acceptance.

Secrets and credentials

Secrets are not audit evidence. Do not place API keys, bearer tokens, private keys, or raw credentials in audit events, PO bundles, replay fixtures, or lineage annotations. Record key ids, signature ids, public-key fingerprints, and verification results instead.

Hard rule. If a replay needs a secret, the design is wrong. Replay may need a signed statement that a secret-backed service produced a value, but not the secret itself.

Abuse cases to test

- Oversized event that attempts to exhaust canonicalization memory.
- Event with duplicate semantic fields in different encodings.
- Bundle that reuses a valid witness from a different step id.
- Descriptor downgrade to an older checker version.
- Recovery event without authorized signature.
- Stutter claim that hides behavior change behind bookkeeping.
- Memory compression that deletes proof-relevant evidence.

Memory and Lineage

Memory can be compressed. Lineage cannot disappear.

Content-addressed memory

Use roots. Do not copy whole memory into every state.

Listing 19: Memory operation witness.

```
@dataclass(frozen=True)
class MemoryWitness:
    prev_root: str
    next_root: str
    operation: str          # retain, compress, summarize, migrate, forget
    deterministic: bool
    lineage_extended: bool
    audit_reconstructible: bool
    entropy_claim: str | None
```

Lawful compression

A compression operation is lawful only when it preserves the evidence required by the constitution. A shorter summary is not automatically lawful.

Failure mode. The dangerous compression is the one that destroys the evidence needed to prove it was lawful.

Lineage operations

Allowed lineage operations:

- extend;
- annotate;
- branch with attribution;
- cryptographically commit;

- summarize with reconstruction evidence.

Forbidden lineage operations:

- erase;
- silently overwrite;
- truncate ancestry;
- merge histories without provenance;
- compress beyond audit.

Audit and Replay

Audit is not telemetry. Audit is the reconstruction path.

Audit event

Listing 20: Audit event shape.

```
{
  "schema": "scc.audit.v1",
  "step_id": "2026-04-29T12:00:00Z:node-a:000001",
  "prev_audit_hash": "...",
  "event_hash": "...",
  "prev_state_hash": "...",
  "next_state_hash": "...",
  "po_hash": "...",
  "mode": "Exact",
  "checker_version": "scc-checker-0.1.0",
  "build_id": "build:abc123",
  "descriptor_id": "theta:prod-a"
}
```

Replay harness

Listing 21: Replay harness.

```
def replay(initial_state, recorded_events, env):
    state = initial_state
    produced = []
    for event in recorded_events:
        state_next, witnesses = seven_stage_transition(state, event, env)
        po = build_bundle(state, event, state_next, witnesses, env)
        if not accepts_bundle(po, state, state_next, env):
            raise ReplayFailure(event["step_id"])
        produced.append((state_next, po))
        state = state_next
    return produced
```

Replay requirements

- Same initial state.
- Same event bytes.
- Same build id.

- Same numerical mode.
- Same execution descriptor.
- Same checker version or a declared migration proof.

Shipping gate. A production release needs at least one replay test for every critical scenario: normal step, rejected step, halt, stutter, safe refinement, memory compression, governance update, and recovery meta-path.

Storage, Transactions, and Concurrency

The audit chain only means something when it is committed with the state it describes. Treat storage as part of the compliance boundary.

Atomic commit rule

The accepted state, PO bundle, and audit event must commit together. The output must wait until that commit succeeds.

Listing 22: Atomic accepted-step commit.

```
def commit_accepted_step(store, candidate_state, po, output_ref):
    po_hash = hash_bundle(po)
    with store.transaction() as tx:
        tx.insert_state(po.next_state_hash, candidate_state)
        tx.insert_bundle(po_hash, po)
        tx.insert_audit_event(po.audit_event)
        tx.compare_and_swap_head(
            expected_prev=po.prev_state_hash,
            next_state=po.next_state_hash,
            next_audit=po.audit_event["next_audit_hash"],
        )
        tx.insert_output_ref(po.step_id, output_ref)
    return po_hash
```

Hard rule. No accepted output may be emitted from an uncommitted candidate. If commit fails, the output is rejected even when the checker accepted the bundle.

Idempotency and duplicate steps

Case	Behavior
Same `step_id`, same event hash, same PO hash	Return the committed result. This is a retry.
Same `step_id`, different event hash	Reject and alert. This is an identity collision or tampering attempt.
Same event hash, different step id	Accept only if the descriptor allows replayed input. Otherwise reject.
Commit after checker version changed	Reject unless descriptor declares a migration proof.

Concurrency model

Start with one writer per lineage head. If multiple writers are required, use compare-and-swap on the head hash and replay losing candidates against the new head.

1. Read current head state hash.
2. Build candidate from that exact head.
3. Build and check PO bundle.
4. Compare-and-swap the head from old hash to next hash.
5. If the swap fails, discard the candidate and retry from the new head.

Crash recovery

On boot, the runtime verifies the audit chain from the last trusted checkpoint to the current head. Any orphaned state, bundle, or output reference is quarantined until a replay repair process classifies it.

Shipping gate. A storage implementation is not hardened until a forced-crash test proves that no output can exist without a committed audit event and no audit event can point to a missing state.

Refinement, Stuttering, and ρ vs. ρ_b

The full abstraction ρ is for accountability. The normalized projection ρ_b is for behavioral simulation.

Why this matters

A bookkeeping step can update audit hash and lineage while leaving behavior unchanged. Under ρ, the state changed. Under ρ_b, the behavior may be unchanged. That is why SCC separates the two maps.

Step mode classifier

Listing 23: Step classification.

```
def classify_step(prev_impl, next_impl, abstract_next, env) -> StepMode | None:
    prev_norm = rho_b(prev_impl)
    next_norm = rho_b(next_impl)
    abs_norm = project_abstract(abstract_next)

    exact = next_norm == abs_norm
    stutter = next_norm == prev_norm
    safe_refined = safer_or_equal(next_norm, abs_norm, env.safe_order)

    modes = [exact, stutter, safe_refined]
    if sum(bool(x) for x in modes) != 1:
        return None
    if exact:
        return StepMode.EXACT
    if stutter:
        return StepMode.STUTTER
    return StepMode.SAFE_REFINED
```

Decision tree

1. Does normalized successor equal the induced abstract successor? Classify as Exact.
2. Else, does normalized successor equal normalized predecessor? Classify as Stutter.
3. Else, is normalized successor safer than the abstract successor under the declared preorder? Classify as SafeRefined.
4. Else, reject the step.
5. If more than one mode applies, reject until the model makes precedence explicit.

Failure mode. Do not define stuttering on the full abstraction map. That freezes audit and lineage, which is wrong. Define behavioral stuttering on ρ_b.

Federation and Quorum Safety

Federation is not a day-one feature. Add it after the single-node kernel is solid.

Federation prerequisites

- Authenticated signatures.
- Decision-instance ids.
- One-signature-per-instance rule for honest nodes.
- Quorum size and fault bound.
- Cross-shard lineage propagation.
- Halt propagation protocol.
- Treaty and hard-clause propagation rules.

Certificate check

Listing 24: Quorum certificate check sketch.

```
def verify_certificate(cert, instance_id, public_keys, quorum_size):
    if cert.instance_id != instance_id:
        return False
    signers = set()
    for sig in cert.signatures:
        if sig.signer in signers:
            return False
        if not verify_signature(public_keys[sig.signer], cert.value, sig):
            return False
        signers.add(sig.signer)
    return len(signers) >= quorum_size
```

Federation warnings

- Quorum safety is not liveness.
- Authentication is not optional.
- Cross-shard lineage must be monotone.
- Local refinement is not automatically global refinement.

- Halt propagation must be bounded and explicit.

Deployment Readiness

Do not call a deployment SCC-compliant unless these exist.

Artifact	Minimum acceptance test
Execution descriptor	Declares build id, task mode, numerical mode, authority stream, and active constitution.
Hard-clause registry	Machine-readable active clauses with versioning and tests.
State schema	All coordinates typed, serialized, and hashable.
Canonical event spec	Equivalent events produce identical bytes across machines.
PO schema	Required fields and witness refs are documented and validated.
Checker	Rejects malformed, contradictory, stale, or incomplete bundles.
Audit chain	Hash chain verifies from genesis to current state.
Replay harness	Reproduces critical traces deterministically.
Lineage tools	Ancestry can be reconstructed without privileged oral history.
Halt and recovery	Halt is absorbing; recovery path is explicit and auditable.
TCB ledger	Lists checker, serializer, crypto, runtime, compiler, storage, and network assumptions.
Stress report	Includes negative tests, not only happy-path demonstrations.

Shipping gate. Production readiness requires evidence. A working demo is not evidence by itself.

Observability, Operations, and Incident Response

Telemetry helps operators see the runtime. It does not replace audit, witnesses, or proof obligations.

Operational metrics

Metric	Meaning	Alert when
`scc_checker_reject_total`	Count of rejected PO bundles.	Sudden spike or repeated same reason.
`scc_halt_total`	Entries into failure mode 3.	Any production halt.
`scc_audit_gap_total`	Audit chain verification gaps.	Greater than zero.
`scc_replay_failure_total`	Recorded traces that do not replay.	Greater than zero after release.

`scc_lineage_gap_total`	Missing or non-reconstructible ancestry.	Greater than zero.
`scc_po_build_seconds`	Bundle construction latency.	Exceeds budget and causes queueing.
`scc_checker_seconds`	Checker latency.	Exceeds hot-path budget.

Logs vs. audit

Record type	Use
Audit event	Reconstructs accepted state transition and verifies hash chain.
PO bundle	Provides machine-checkable step evidence.
Witness	Supplies typed, minimal evidence to the checker.
Telemetry log	Helps humans debug performance and operations. Not legal evidence by itself.
Incident note	Explains operator decisions during halt, recovery, or rollback.

Runbook: checker unavailable

1. Stop ordinary output emission.
2. Enter halt or rejection path according to the constitution.
3. Record a canonical incident event.
4. Verify checker binary, config, descriptor, and schema registry.
5. Replay the last accepted trace from the latest checkpoint.
6. Resume only through an authorized recovery path.

Runbook: audit mismatch

1. Freeze the affected lineage head.
2. Compare state hash, event hash, PO hash, and audit hash against the last signed checkpoint.
3. Quarantine orphaned records.
4. Replay from the last trusted checkpoint.
5. Record repair or remain halted. Do not patch the hash chain by hand.

Incident record

Every production halt should produce an incident record with the step id, descriptor id, checker version, previous state hash, observed failure, operator, recovery authority, replay result, and final disposition.

Performance Guide

SCC adds overhead. That is not a bug. The goal is to know where the overhead is and keep it lawful.

Hot-path costs

- Canonical serialization.
- Hash updates.

- Bundle construction.
- Checker evaluation.
- Drift measurement.
- Lineage writes.
- Replay-safe storage commits.

Safe optimizations

Optimization	Allowed when	Danger
Incremental hashing	Hash domains and segment boundaries are explicit.	Accidentally changing audit semantics.
Content-addressed memory	Roots preserve reconstruction paths.	Losing evidence behind a summary.
Batch verification	No accepted output depends on unchecked critical fields.	Letting illegal steps run before rejection.
Sparse vector clocks	Missing entries have a canonical zero rule.	Divergent interpretations across nodes.
Precomputed witnesses	Inputs, build id, and descriptor are pinned.	Reusing stale evidence.
Async cold-path analysis	Hard clauses are still checked synchronously.	Moving legality out of the hot path.

Do not optimize these away

- Protected-coordinate check.
- Hard-clause check.
- PO checker gate.
- Audit hash update.
- Halt absorption.
- Event canonicalization.

Failure mode. The fastest illegal path is still illegal. Performance does not weaken a hard clause.

CI/CD and Release Gates

A hardened SCC repository rejects unsafe changes before they reach production. The release process is part of the runtime story.

Pre-merge gates

1. Schema validation and golden-vector checks.
2. Unit tests for serializers, hash domains, and state invariants.
3. Negative corpus tests against the checker.

4. Replay determinism tests for all critical traces.
5. Static check that checker code has no forbidden imports.
6. TCB ledger diff review when trusted dependencies change.
7. Documentation check for descriptor, runbook, and migration notes.

Release manifest

Every release should produce a signed manifest.

Listing 25: Release manifest shape.

```
schema: scc.release_manifest.v1
release_id: scc-runtime-0.1.0
build_id: build:abc123
source_commit: git:...
checker_version: scc-checker-0.1.0
state_schema: scc.state.v1
event_schema: scc.event.v1
po_schema: scc.po.v1
audit_schema: scc.audit.v1
numerical_mode: deterministic-fp-v1
negative_corpus_hash: ...
replay_suite_hash: ...
tcb_ledger_hash: ...
```

CI sketch

Listing 26: CI gate sketch.

```
name: scc-kernel-gates
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python -m unittest discover -s tests -v
      - run: make replay
      - run: make verify
      - run: python tools/check_forbidden_imports.py src/scc_kernel/checker.py
```

Release rejection rules

Reject a release when any of these is true.

- A replay fixture changed without a declared reason.
- The checker version changed without negative corpus review.
- The descriptor references a schema not present in the registry.
- The TCB ledger changed without owner approval.
- Audit and state commit are no longer atomic.
- A production path can emit output before accepted commit.

Test Harness

A serious SCC implementation needs hostile tests. Friendly tests are not enough.

Minimum negative corpus

- Missing PO field.
- Contradictory PO mode.
- Invalid governance simplex.
- Protected-coordinate mutation.
- Risk threshold crossed without halt.
- Audit hash mismatch.
- Non-canonical event encoding.
- Lineage erasure.
- Stutter that changes behavior.
- SafeRefined claim without safer-direction witness.
- Quorum certificate with duplicate signer.
- Replay under wrong build id.

Property test examples

Listing 27: Property-style tests.

```
def test_halt_absorption(prev_state, env):
    halted = replace(prev_state, failure_mode=3)
    nxt, witnesses = seven_stage_transition(halted, benign_event(), env)
    po = build_bundle(halted, benign_event(), nxt, witnesses, env)
    assert accepts_bundle(po, halted, nxt, env)
    assert nxt.failure_mode == 3

def test_protected_coordinate_cannot_change(prev_state, env):
    malicious = replace(prev_state, protected_root="evil")
    po = fake_bundle_claiming_ok(prev_state, malicious)
    assert not accepts_bundle(po, prev_state, malicious, env)

def test_replay_is_deterministic(trace, env):
    run1 = replay(trace.initial_state, trace.events, env)
    run2 = replay(trace.initial_state, trace.events, env)
    assert [po.next_state_hash for _, po in run1] == [po.next_state_hash for _, po in run2]
```

Stress scenarios

1. Self-modification attempt.
2. Governance capture attempt.
3. Memory compression under audit pressure.
4. Shard-coupling drift spike.
5. Replayed event with modified bytes.
6. Recovery attempt from halt without authority.
7. Federation conflict certificate.

8. Stuttering abuse: bookkeeping step hides behavior change.

Decision Trees

Use these as code-review checklists.

Incoming step

1. Is the raw input converted to a canonical event? If no, reject.
2. Is the current state admissible? If no, halt or recover through meta-path.
3. Are hard clauses active and readable by the runtime? If no, halt.
4. Does the seven-stage transition produce a candidate state? If no, reject.
5. Does the PO bundle cover every required field? If no, reject.
6. Does the checker accept? If no, reject or halt.
7. Is audit committed atomically with the accepted state? If no, reject.
8. Emit output only after acceptance.

Memory operation

1. Is the operation typed? If no, reject.
2. Does it preserve lineage? If no, reject.
3. Can audit reconstruct lawfulness? If no, reject.
4. Does it alter protected state? If yes, require authorized override.
5. Does it update memory root and audit event canonically? If no, reject.

Governance update

1. Is the proposal source legitimate? If no, use fallback identity update.
2. Is the update projected into the simplex? If no, reject.
3. Is drift bounded and recorded? If no, reject.
4. Are hard clauses preserved? If no, reject.

Recovery from halt

1. Is current mode 3? If no, ordinary execution may continue.
2. Is there an authorized recovery meta-path? If no, remain halted.
3. Is the recovery event canonical and signed? If no, remain halted.
4. Does recovery preserve lineage and audit? If no, remain halted.
5. Does the checker accept the recovery bundle? If no, remain halted.

Mechanization Priorities

The practical priority is not to formalize everything at once. Formalize the pieces that carry the compliance story.

1. PO bundle as a typed record.

2. Checker soundness.
3. Step mode exclusivity.
4. Halt absorption.
5. Protected-coordinate non-interference.
6. Replay determinism.
7. Pointwise trace compliance.
8. Quorum safety, after the single-node proofs are stable.

Shipping gate. The highest-value mechanization target is checker soundness. Without it, accepted runtime booleans do not automatically become mathematical predicates.

Code Review Questions

Ask these in every review.

1. Can this path emit output without a checked PO bundle?
2. Can ordinary code change protected state?
3. Can a failed checker be ignored?
4. Can audit and state commit separately?
5. Can replay depend on a live service?
6. Can a governance update skip projection?
7. Can a halted state resume through ordinary execution?
8. Can two step modes hold at once?
9. Can lineage be overwritten by migration?
10. Can a performance optimization weaken a hard clause?

    If the answer is yes, the code is not ready.

Common Bad Patterns

Bad pattern	Why it fails	Fix
Policy prompt as constitution	Prompts are not typed transition law.	Put hard clauses in a machine-readable registry.
Log-only compliance	Logs do not enforce.	Build PO bundles and a checker.
Best-effort audit	Audit gaps break replay.	Commit audit atomically with state.
Magic safety score	Scores do not prove legality.	Map score to explicit risk law and halt thresholds.
Mutable protected state	Sovereignty membrane is gone.	Gate protected updates through authorized meta-paths.

Unversioned schemas	Replay and review become unstable.	Version every state, event, PO, and checker schema.
Async hard-clause checks	Illegal output may already escape.	Keep hard checks synchronous.
Federation first	Distributed complexity hides kernel bugs.	Harden single-node SCC first.

Builder Deliverables

This section converts the handbook into concrete engineering deliverables.

Minimum repository deliverables

Deliverable	Acceptance condition
`src/scc_kernel/state.py`	Immutable state record, canonical serializer, state hash golden vector.
`src/scc_kernel/events.py`	Canonical event envelope and event hash golden vector.
`src/scc_kernel/po.py`	Typed PO bundle and bundle hash.
`src/scc_kernel/checker.py`	Deterministic fail-closed checker with negative tests.
`src/scc_kernel/runtime.py`	StepResult output gate and no output before commit.
`src/scc_kernel/store.py`	Atomic accepted-step commit and idempotency checks.
`src/scc_kernel/replay.py`	Deterministic replay from fixtures.
`tests/test_negative_corpus.py`	Covers protected mutation, audit mismatch, stale checker, invalid simplex, and bad mode.
`docs/tcb_ledger.md`	Lists trusted dependencies, owners, versions, and test evidence.
`docs/runbook.md`	Covers checker outage, audit mismatch, halt, and recovery.

Pull request checklist

1. Does this change alter a schema, hash domain, descriptor, checker, or serializer?
2. If yes, did the release manifest and TCB ledger change?
3. Did new behavior add a replay fixture?
4. Did new rejection logic add a negative test?
5. Can any path emit output before PO acceptance and storage commit?
6. Can this path read live time, randomness, environment, or network in replay-sensitive code?
7. Does this change preserve protected-coordinate isolation?

Review posture

Review the SCC kernel like a small security-critical database and verifier, not like ordinary application glue. Simplicity is a feature. The best kernel code is plain enough that a tired reviewer can see the failure path.

Implementation Skeleton

This is a compact map of the files a first reference implementation should contain.

Listing 28: Reference implementation layout.

```
scc_kernel/
  state.py              # SCCState, hashes, serializers
  constitution.py       # hard clauses and active clause registry
  events.py             # canonical event encoding
  audit.py              # audit event and hash chain
  po.py                 # POBundle schema
  checker.py            # fail-closed checker
  transition.py         # seven-stage transition
  governance.py         # simplex projection and legitimacy gate
  risk.py               # risk update, escalation, halt
  memory.py             # memory operations and lineage witnesses
  refinement.py         # rho, rho_b, step classification
  replay.py             # deterministic replay harness
  tests/
    test_negative_corpus.py
    test_replay.py
    test_halt.py
    test_protected_state.py
    test_governance.py
  docs/
    tcb_ledger.md
    deployment_descriptor.yaml
    hard_clauses.yaml
```

Descriptor example

Listing 29: Execution descriptor example.

```
schema: scc.descriptor.v1
build_id: build:abc123
numerical_mode: deterministic-fp-v1
task_mode: production-agent
checker_version: scc-checker-0.1.0
active_constitution: scc-constitution-2026-04-26
hard_clause_set: hard-clauses:v1
audit_hash_domain: SCC-AUDIT-V1
po_schema: scc.po.v1
risk_model: risk:v1
governance_model: governance:v1
replay_required: true
```

Glossary

Term	Engineering meaning
SCC	Constitutional-operational kernel for lawful cognitive-state transitions.
Hard clause	Rule that ordinary execution cannot weaken or bypass.

Protected coordinate	State coordinate that ordinary operation cannot mutate.
PO bundle	Machine-checkable witness record for a step.
Checker	Deterministic gate that accepts or rejects a bundle.
Audit hash	Canonical hash chain over accepted events and state changes.
Lineage	Reconstructible ancestry of state, memory, and governance.
ρ	Full abstraction map for accountability.
ρ_b	Normalized projection for behavioral stuttering and simulation.
Stutter	Implementation step with unchanged normalized behavior but possibly advanced audit or lineage.
SafeRefined	Implementation successor that is safer than the ordinary abstract successor under the declared preorder.
Halt absorption	Once mode 3 is entered, ordinary execution cannot silently leave it.
Execution descriptor	Versioned declaration of build id, schema ids, checker version, numerical mode, active constitution, and runtime assumptions.
Release manifest	Signed release record tying source commit, schemas, checker, replay suite, negative corpus, and TCB ledger together.
Atomic commit	Storage rule requiring state, PO bundle, audit event, and output reference to commit as one accepted step.
TCB	Trusted computing base: checker, serializer, crypto, runtime, compiler, storage, and assumptions.

Hardening Changelog

This hardened builder edition adds engineering material intended to make the handbook easier to implement and review.

- Added a builder quickstart with repository layout, make targets, and first runnable slice.
- Added runtime boundary and TCB hardening guidance.
- Added schema, versioning, migration, and compatibility rules.
- Added atomic storage, idempotency, concurrency, and crash-recovery rules.
- Added security and abuse-hardening threat model.
- Added observability, runbooks, incident records, and production metrics.
- Added CI/CD gates, release manifest shape, and release rejection rules.
- Added builder deliverables and pull request checklist.
- Added Kernel Fortress handoff protocol: reviewer-ready artifact contents, fixture-to-gate discipline, release evidence rule, and paper-facing wording.
- Added mechanization, federation, numerical-stability, and one-page navigation hardening rules for the Kernel Fortress handoff.

Kernel Fortress Handoff Protocol

Kernel Fortress is the review-grade executable boundary for a minimal SCC kernel. A builder should be able to hand a reviewer a package that runs without product services, accepts golden transitions, rejects named violations, and exposes the trusted computing base. This section is the handoff checklist between the handbook and the artifact.

Shipping gate. A Kernel Fortress handoff is complete only when a reviewer can run the release gate, inspect every fixture, and see which canon article each gate implements. A README is not evidence. A transcript without source is not evidence. A source tree without committed fixtures is not evidence. The package needs all three.

Reviewer-ready artifact contents

Artifact	Builder-ready condition	Reviewer question it answers
Source-of-truth checker	Small deterministic gate, no product logic, no network, no clocks, no randomness.	What exact predicate accepts or rejects the transition?
Reference checker	Independent or simpler implementation for audit and differential inspection.	Does another implementation agree on the committed corpus?
Golden vectors	Exact, Stutter, and SafeRefined accepted fixtures in canonical byte form.	What lawful behavior is accepted?
Negative corpus	Named first-failure fixtures for missing evidence, audit corruption, protected mutation, halt escape, governance corruption, risk violation, lineage erasure, and wrong step mode.	Are failures executable fixtures or merely illustrative examples?
Schemas and descriptors	Versioned carrier schemas and execution descriptor.	Which runtime assumptions and schema versions govern the decision?
TCB ledger	Checker, serializer, crypto, compiler/runtime, storage, and external assumptions named.	What must be trusted for the evidence to mean anything?
Release script	One command that runs vector validation, TCB import scan, source checker tests, reference tests, and manifests.	Can the reviewer reproduce the claim without asking the authors?
CI evidence	Transcript generated by the same release script on a clean runner.	Did the release gate actually execute?

Fixture-to-gate discipline

A case-study table in a paper is not enough. Each row that describes an agent-loop failure should point to a committed fixture. The expected first-failure gate is part of the contract. If a later checker rejects the same fixture at a different first-failure gate, that is a regression unless the manifest and paper explicitly state why the pipeline order changed.

Listing 30: Minimum negative-corpus manifest shape.

```
{
  "schema": "scc.negative_manifest.v1",
```

```
  "cases": [
    {"fixture": "missing_required_field.json",
     "expected_gate": "missing_required_field"},
    {"fixture": "audit_event_hash_bad.json",
     "expected_gate": "audit_event_hash_bad"},
    {"fixture": "risk_threshold_without_halt.json",
     "expected_gate": "risk_threshold_without_halt"}
  ]
}
```

Release evidence rule

Hard rule. Do not claim a checker pass unless the checker ran. If Rust is the source-of-truth checker, a TypeScript pass is useful reference evidence but not a Rust pass. Attach the Rust-equipped CI or local transcript before making the final release claim.

A reviewer-grade transcript should contain the environment versions, release command, golden-vector acceptance, negative-corpus first-failure results, reference-checker results, forbidden-import scan, and release manifest hash. Keep failed or unavailable commands in the transcript. Evidence discipline is stronger than optimism.

Paper-facing wording

When writing about Kernel Fortress, keep the hierarchy clear:

- Canon equals formal authority and claim boundary.
- Handbook equals builder discipline and handoff checklist.
- Kernel Fortress equals executable evidence for the implemented subset.

Use "executable boundary artifact" in the abstract. Save "proof-of-boundary" for the conclusion or final engineering position. This prevents reviewers from reading the artifact as an overbroad proof of safety.

Mechanization, Federation, Numerical Stability, and Overview Handoff

This section closes the four remaining handoff gaps that make a Kernel Fortress package feel fully reviewable rather than merely well documented.

Shipping gate. A 10/10 handoff contains executable evidence, proof-boundary evidence, distributed-boundary evidence, numerical-boundary evidence, and a one-page navigation map. Missing any one of these does not invalidate the kernel, but it weakens reviewer trust.

Mechanization handoff

A builder should not say "mechanized" when the artifact only contains prose. A mechanization handoff has files, theorem anchors, a manifest, and a proof-gate command.

Mechanization item	Required condition	Do not claim

Lean/Coq bridge	Contains no `sorry`, `admit`, `Admitted`, or unledgered `axiom`.	Full canon proof.
Bool-to-Prop bridge	Accepted v1 micro-gate implies Prop-level implemented obligations.	Rust equivalence unless proved.
Seven-stage witnesses	Stage witness record has a Prop-level completeness theorem.	Semantic harmlessness.
Formal gate	`bash scripts/run_formal_gates.sh` runs on a proof-equipped runner.	Local proof pass if Lean/-Coq did not run.

Memory and lineage under compression

Compression is lawful only if lineage remains reconstructible. The stress test must repair hashes first, then test lineage. Otherwise the checker may reject for a superficial hash mismatch before reaching the lineage law.

Listing 31: Compression stress rule.

```
1. Start from a valid golden transition.
2. Change the event to memory_compress with a deterministic compression witness.
3. Recompute event hash, lineage root, audit event, next-state hash, and PO hashes.
4. Assert the repaired transition accepts.
5. Reset lineage to the previous root, repair all hashes again, and assert LineageViolation.
```

Federation handoff

Do not start with federation, but do not leave it as pure prose either. Treat federation as certificates over single-node legality.

Distributed item	**Evidence**	**Boundary**
Quorum certificate	Committee size $n = 3f + 1$, quorum size $q = 2f + 1$, signer set, state root, descriptor hash.	Signature verification remains TCB unless mechanized.
Quorum intersection	Coq arithmetic proof that two quorums overlap in at least $f + 1$ nodes.	Network synchrony remains an assumption.
Cross-shard lineage	Target shard binds source lineage root into its next event.	Does not prove global liveness.
Halt propagation	Halt certificate must propagate within descriptor bound.	Bound depends on network model.

Numerical-stability handoff

Governance and risk arithmetic must be named in the TCB ledger. A descriptor must state numerical mode, maximum vector dimensions, simplex epsilon, risk threshold, and threshold margin. For production-critical deployments, prefer fixed-point arithmetic over binary floating point.

Hard rule. A checker pass without a numerical-mode declaration is incomplete evidence. A risk value within the declared tolerance of the halt threshold should be treated as halt-required unless the deployment uses a verified fixed-point mode.

System overview one-pager

Every release should include a one-page system map with this flow:

Listing 32: System evidence flow.

```
Canon clause -> Handbook rule -> Kernel Fortress gate -> Fixture -> Release transcript -> Reviewer
    claim.
```

The map prevents documentation sprawl. It tells reviewers which document has authority, which artifact has executable evidence, and which claims remain outside the proof.

Final Engineering Position

The scholarly canon tells reviewers what SCC means. This handbook tells builders what to do Monday morning.

Build the checker. Build the PO bundle. Make every state typed. Make every event canonical. Keep audit replayable. Keep lineage reconstructible. Keep protected state protected. Keep halt absorbing. Add optimization only after legality is locked. Add federation only after the single-node kernel is boring.

That is the practical core of SCC.

Copyright and License